

Mastering NumPy

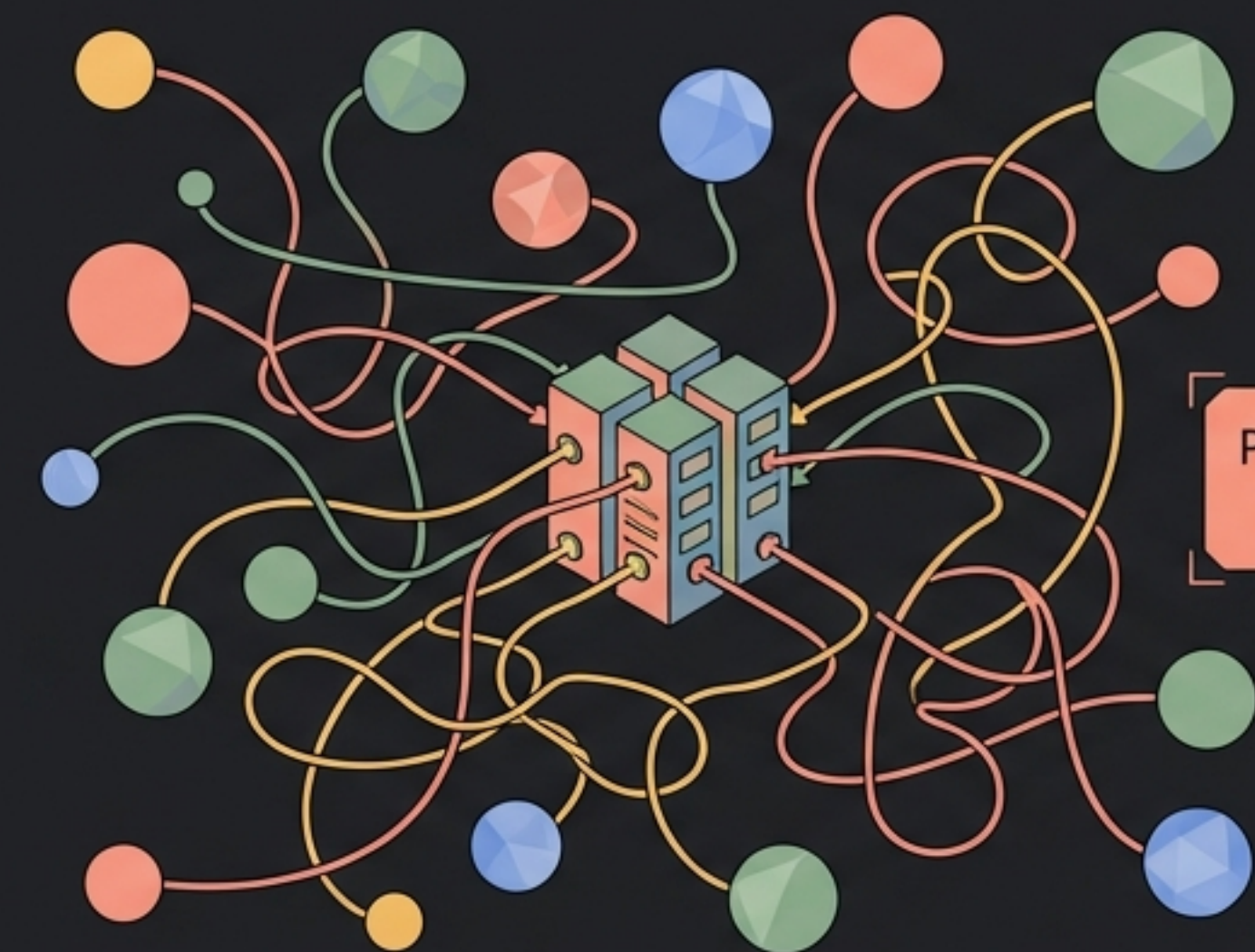
The High-Performance Engine for
Data Science

From Contiguous Memory to Matrix Operations –
A Comprehensive Technical Guide

Based on the curriculum from Sheryians AI School

The Need for Speed: Lists vs. Arrays

Python List (Slow)



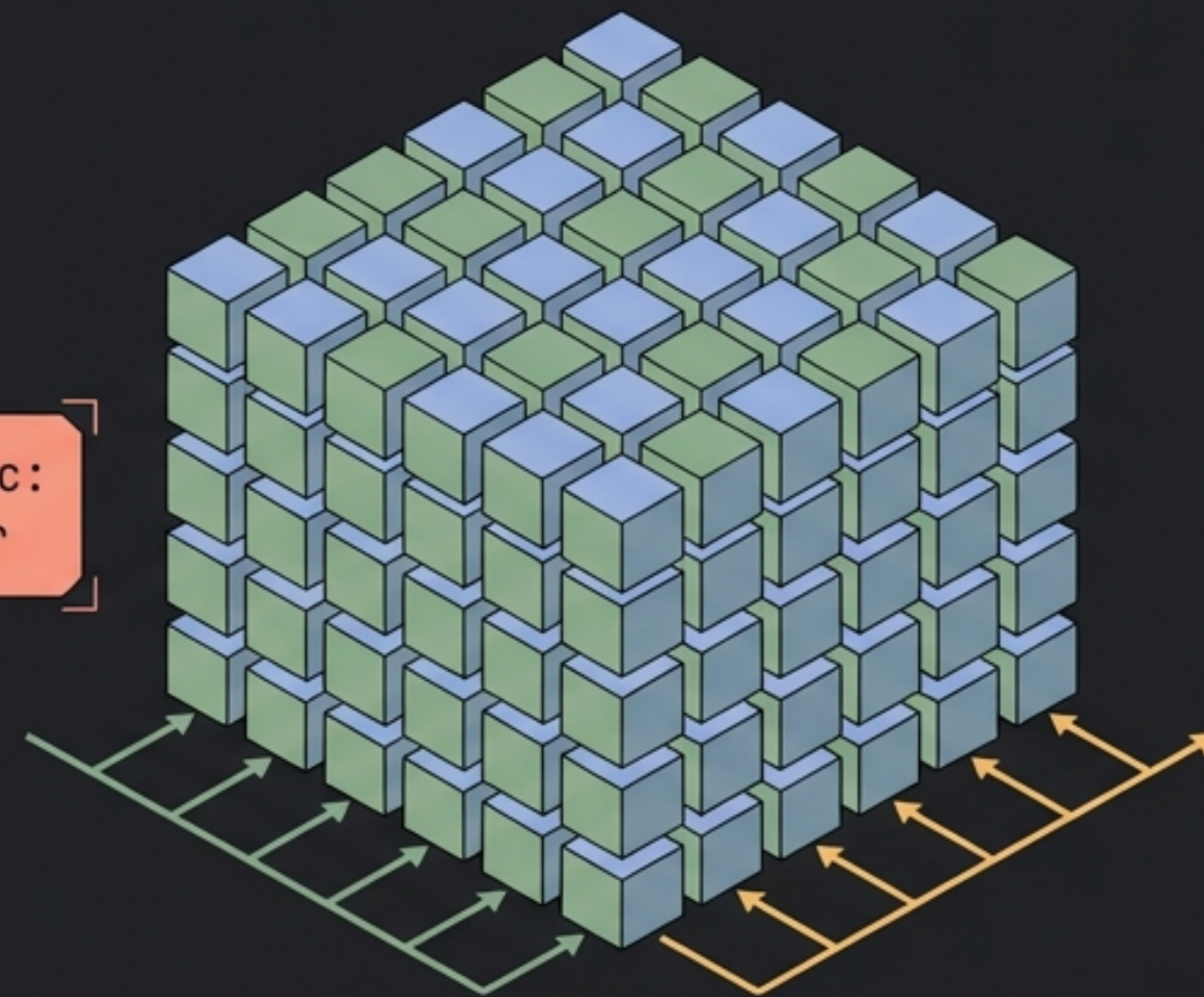
Scattered Memory

Stores References

Flexible Types

Performance Metric:
Up to 50x Faster

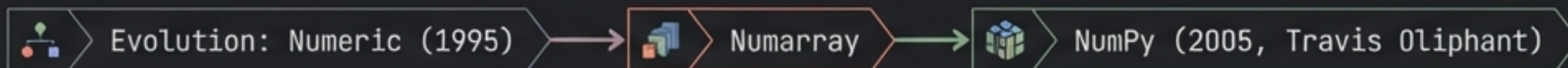
NumPy Array (Fast)



Contiguous Memory (C-Backend)

SIMD CPU Vectorization

Homogeneous Data Only



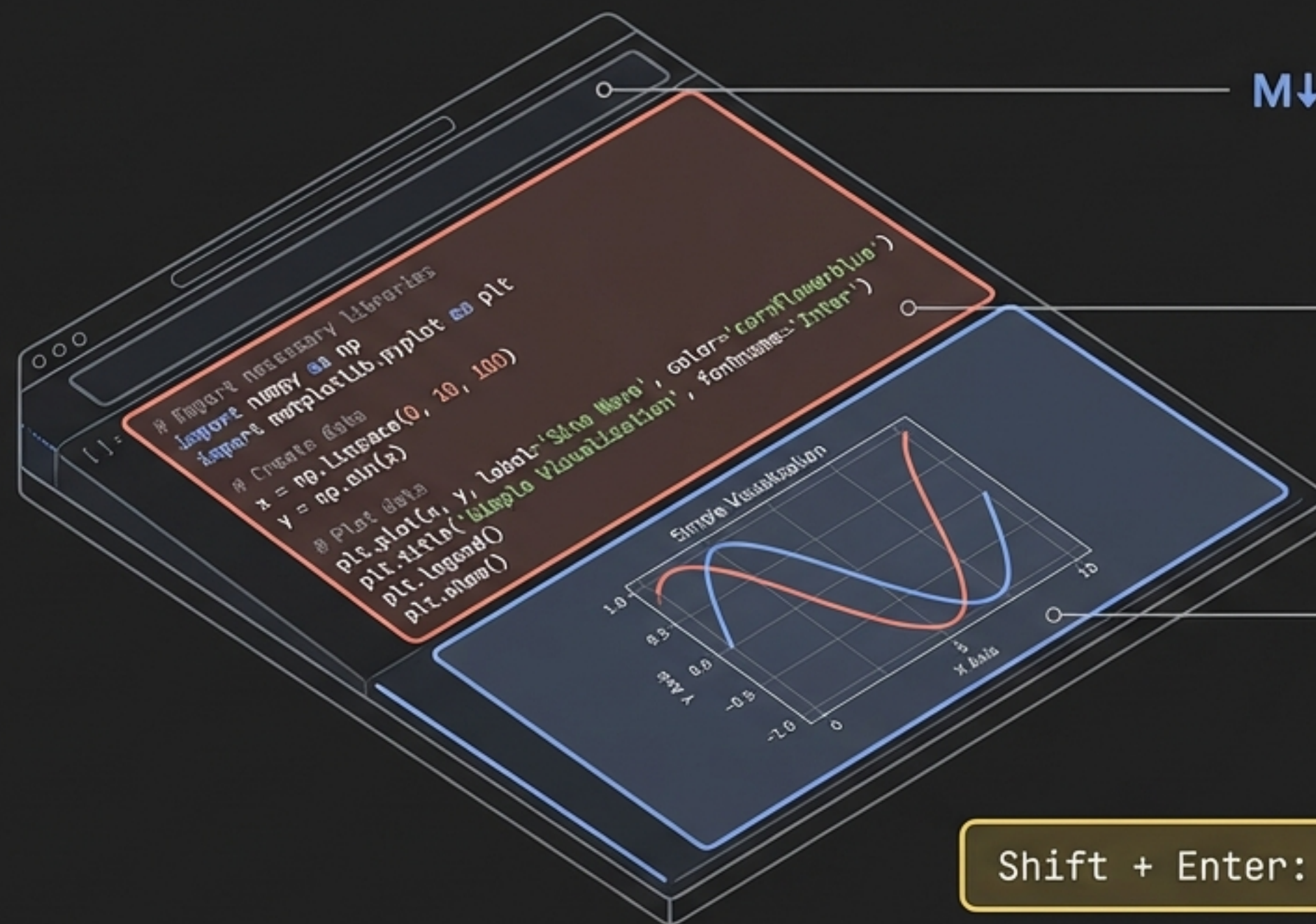
The Canvas: Jupyter Notebooks

The Ecosystem

Anaconda:
Pre-installed libraries
(Batteries Included)

VS Code:
Extension-based,
Developer Workflow

Google Colab:
Cloud-based + Free
GPU Access

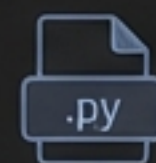


M↓ Markdown Support
(Documentation)

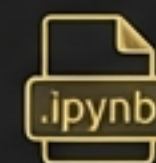
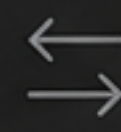
Live Code
(Cell-based execution)

Rich Output
(Visualizations)

Shift + Enter: Run Cell & Advance

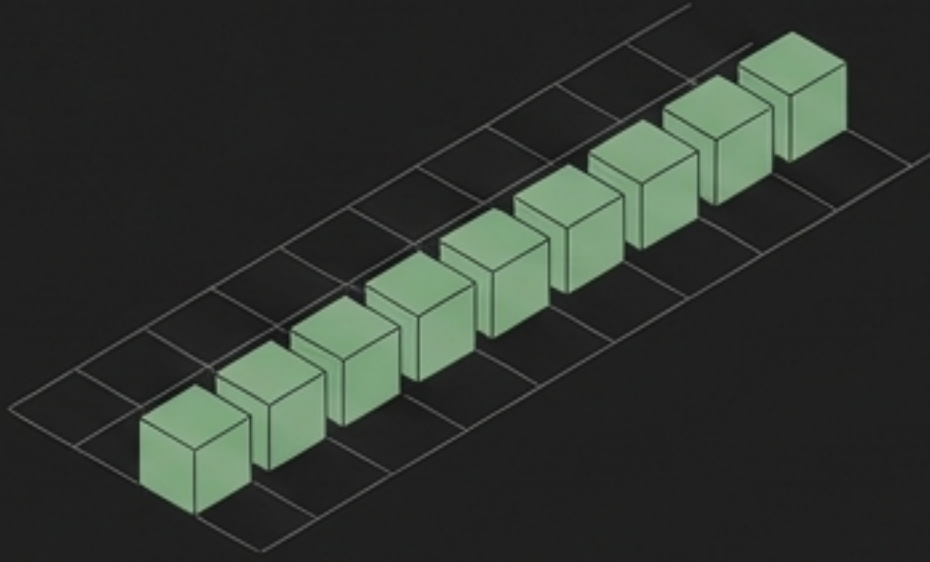


.py
(Sequential Script)

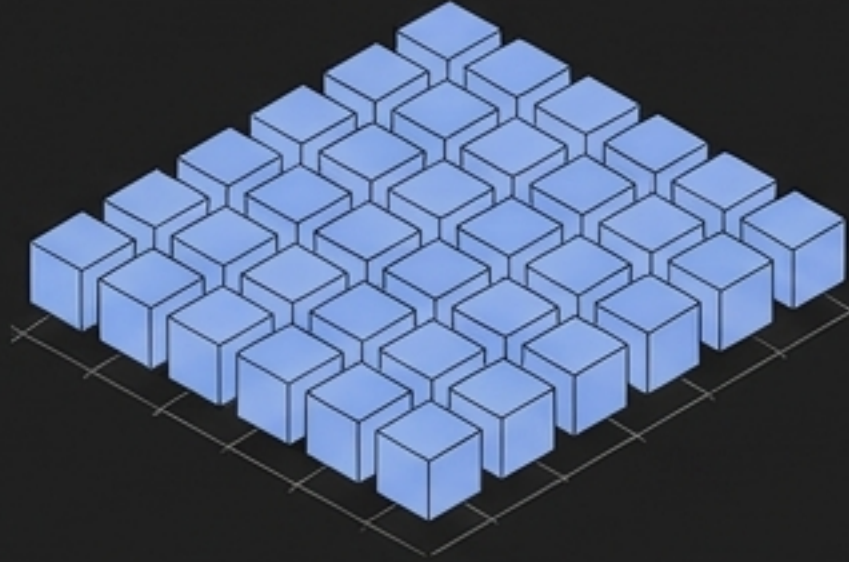


.ipynb
(Interactive Cells)

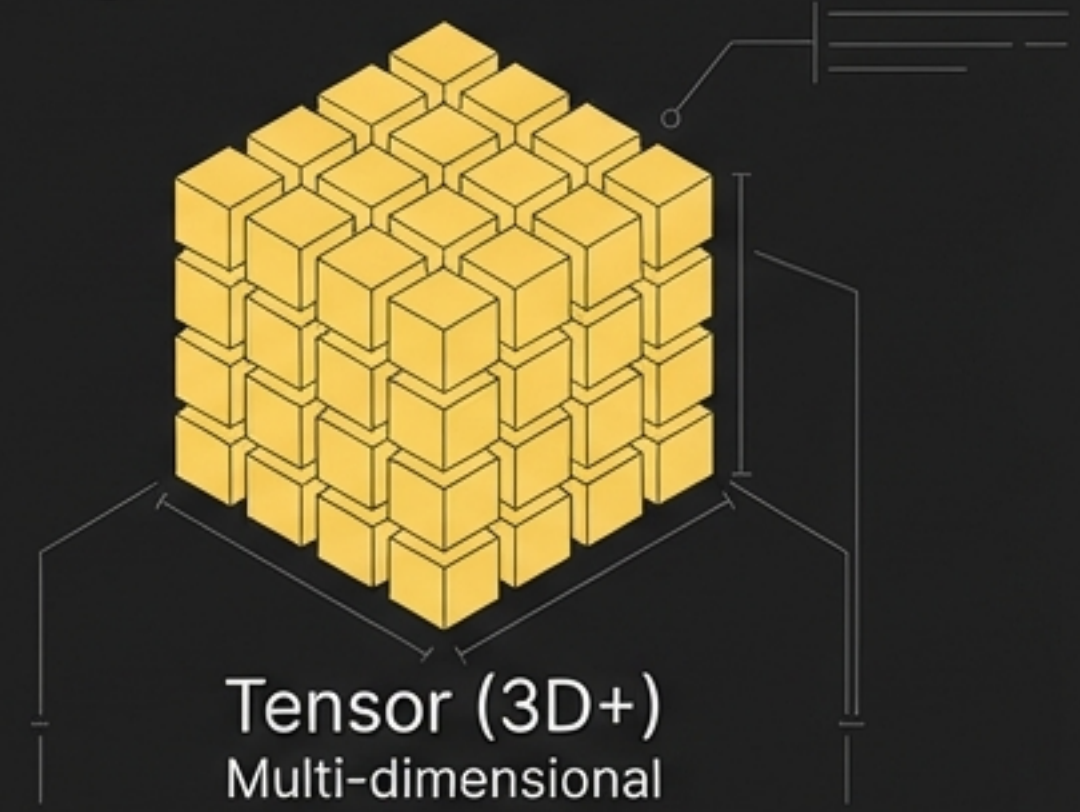
Anatomy of the ndarray



Vector (1D)
Shape: (n,)

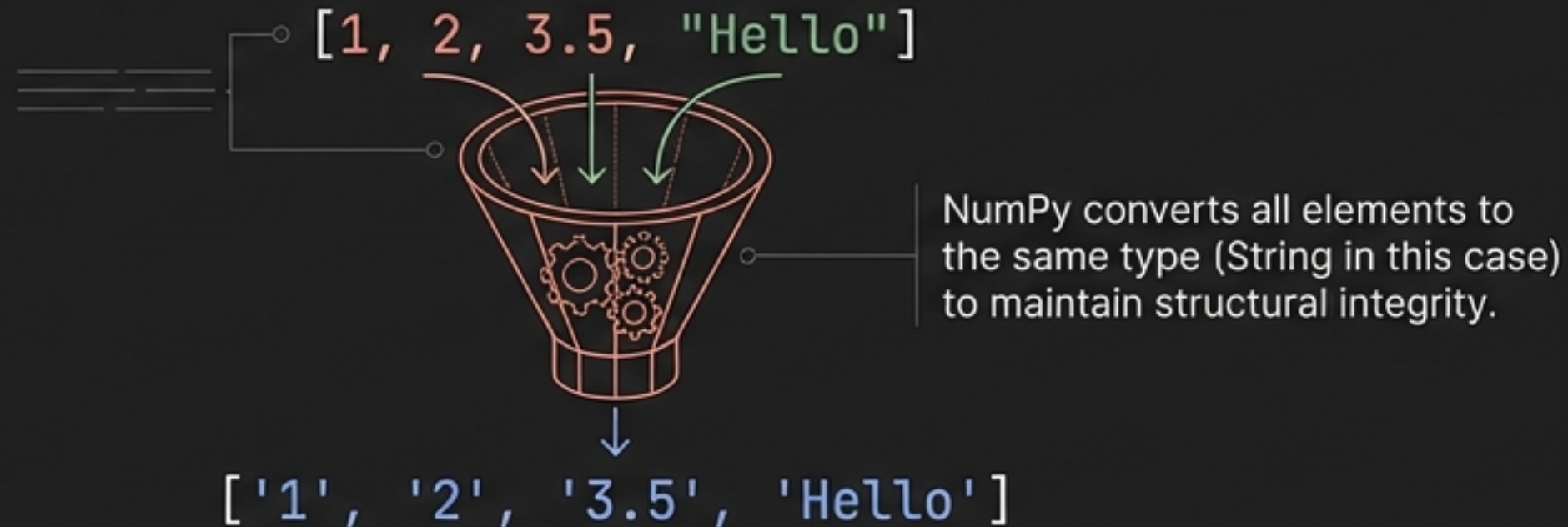


Matrix (2D)
Shape: (rows, cols)



Tensor (3D+)
Multi-dimensional

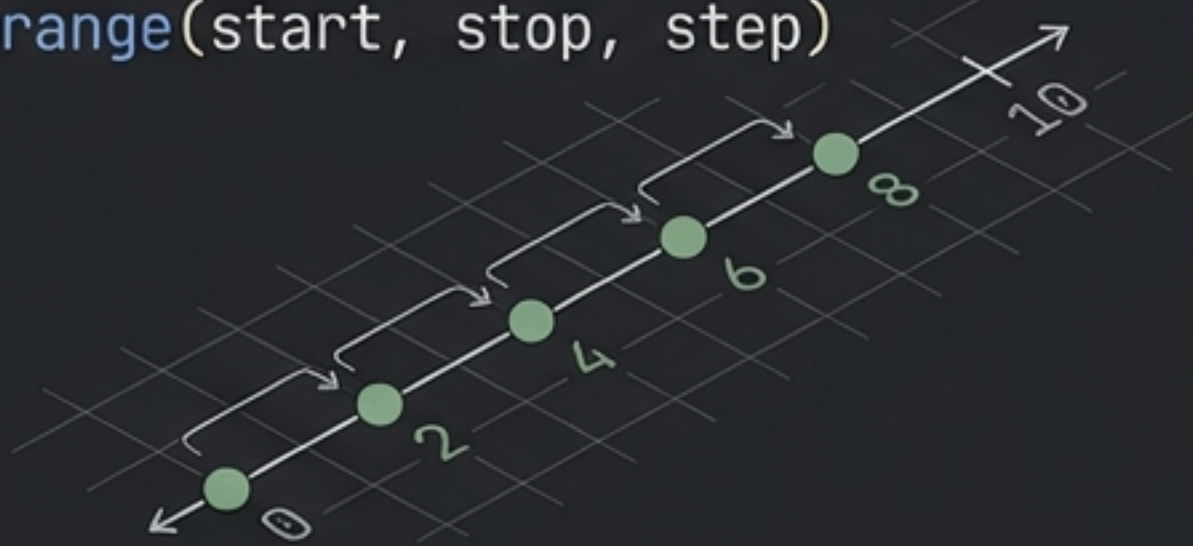
The Homogeneity Rule



Toolkit: Algorithmic Array Generation

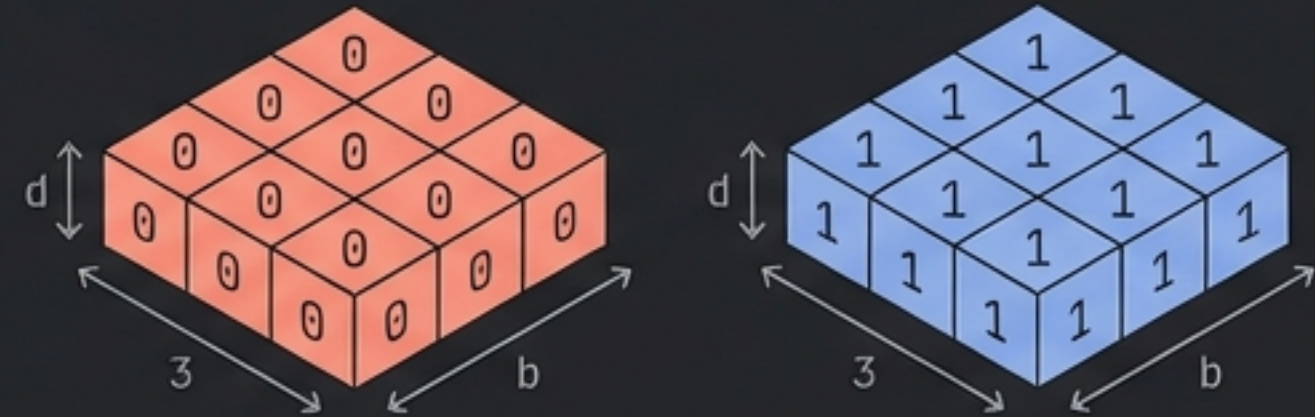
Range Generation

`np.arange(start, stop, step)`



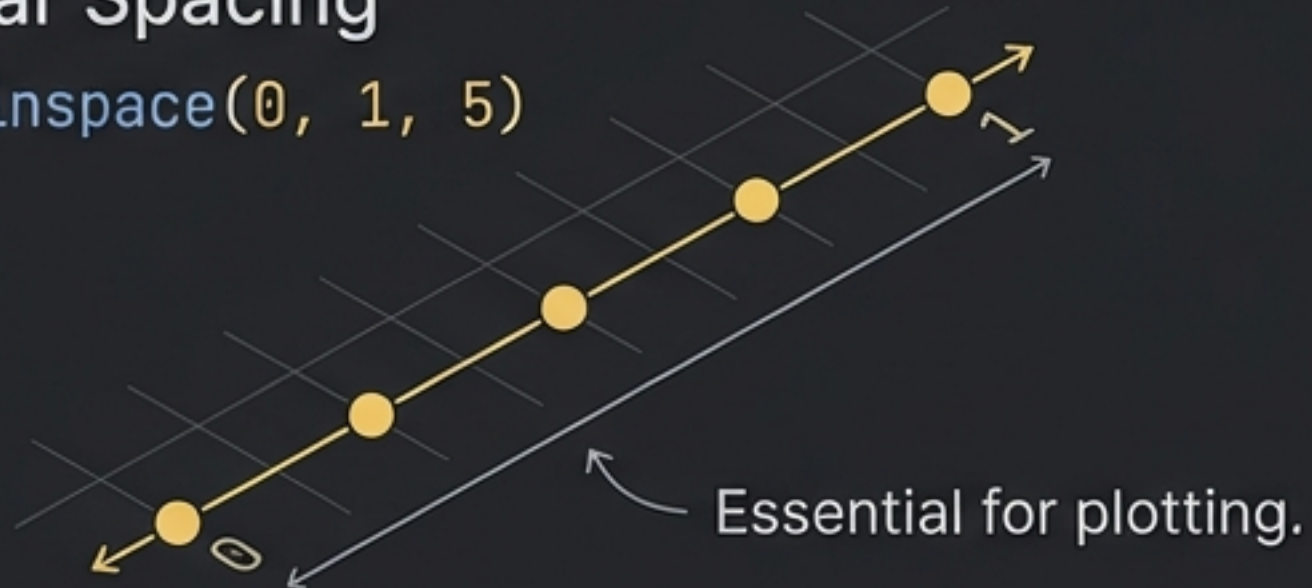
Placeholders

`np.zeros((3,3))` / `np.ones((3,3))`



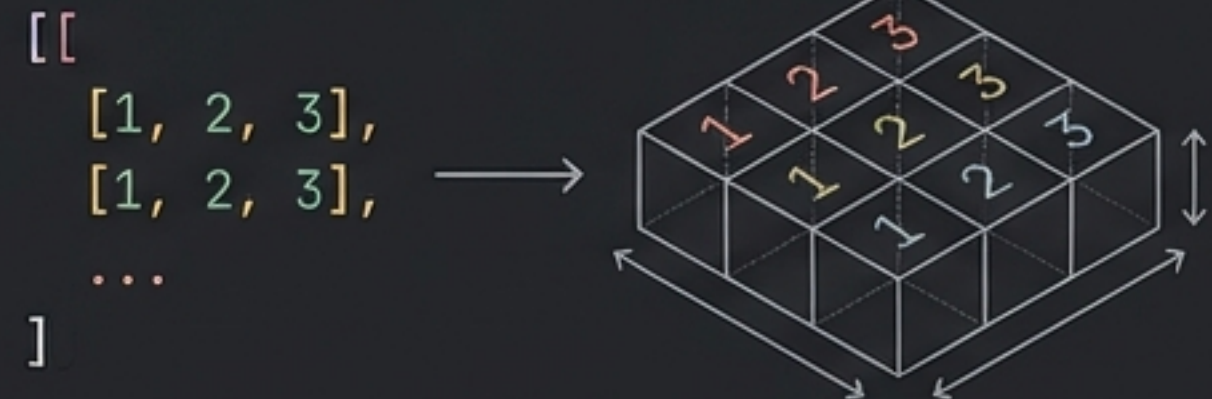
Linear Spacing

`np.linspace(0, 1, 5)`



Manual Creation

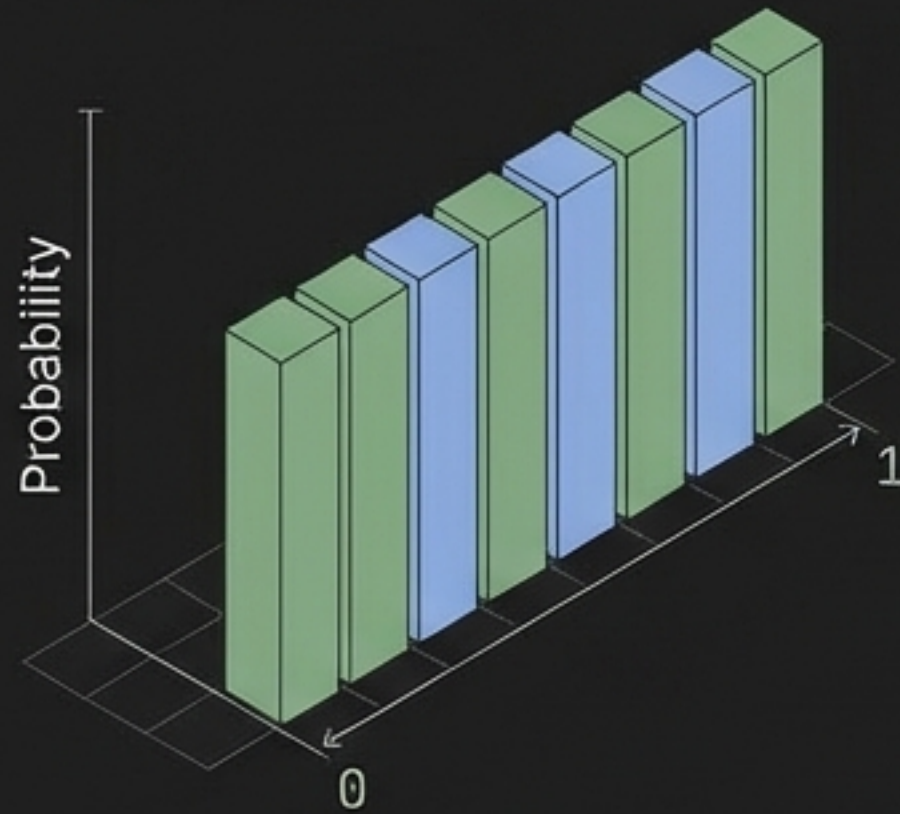
`np.array([list])`



Simulating Reality: Randomness

Uniform Distribution

`np.random.rand()`

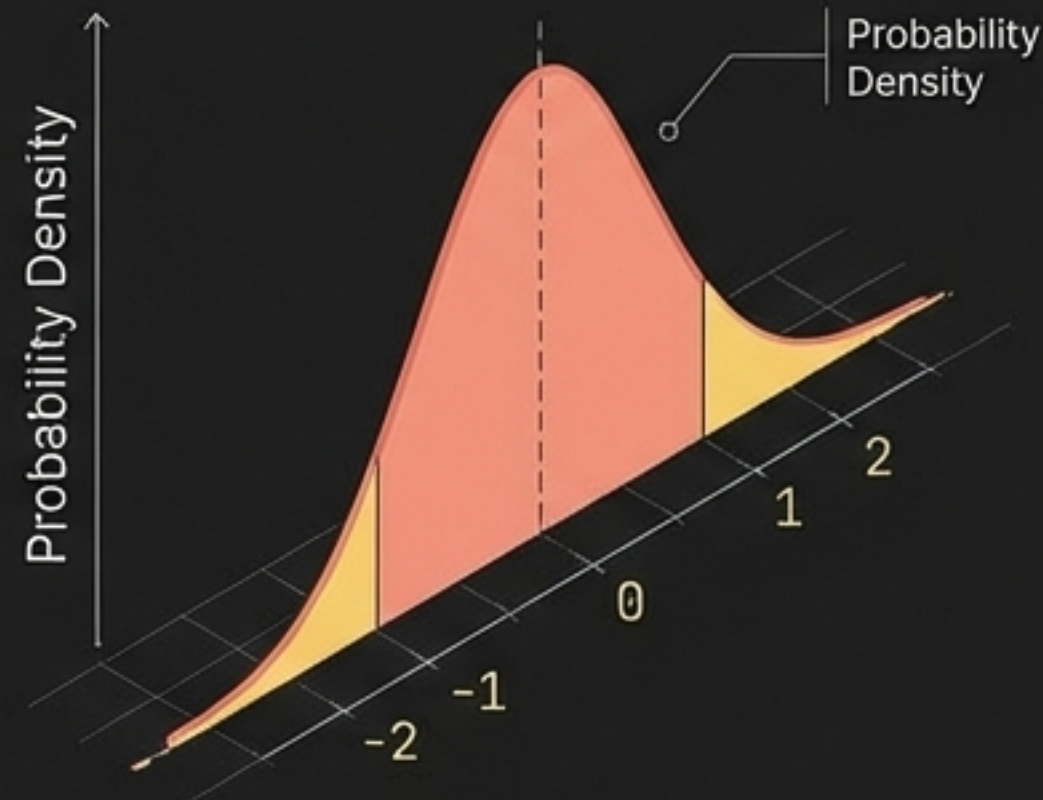


Normalization

Values strictly between 0 and 1.

Standard Normal Distribution

`np.random.randn()`

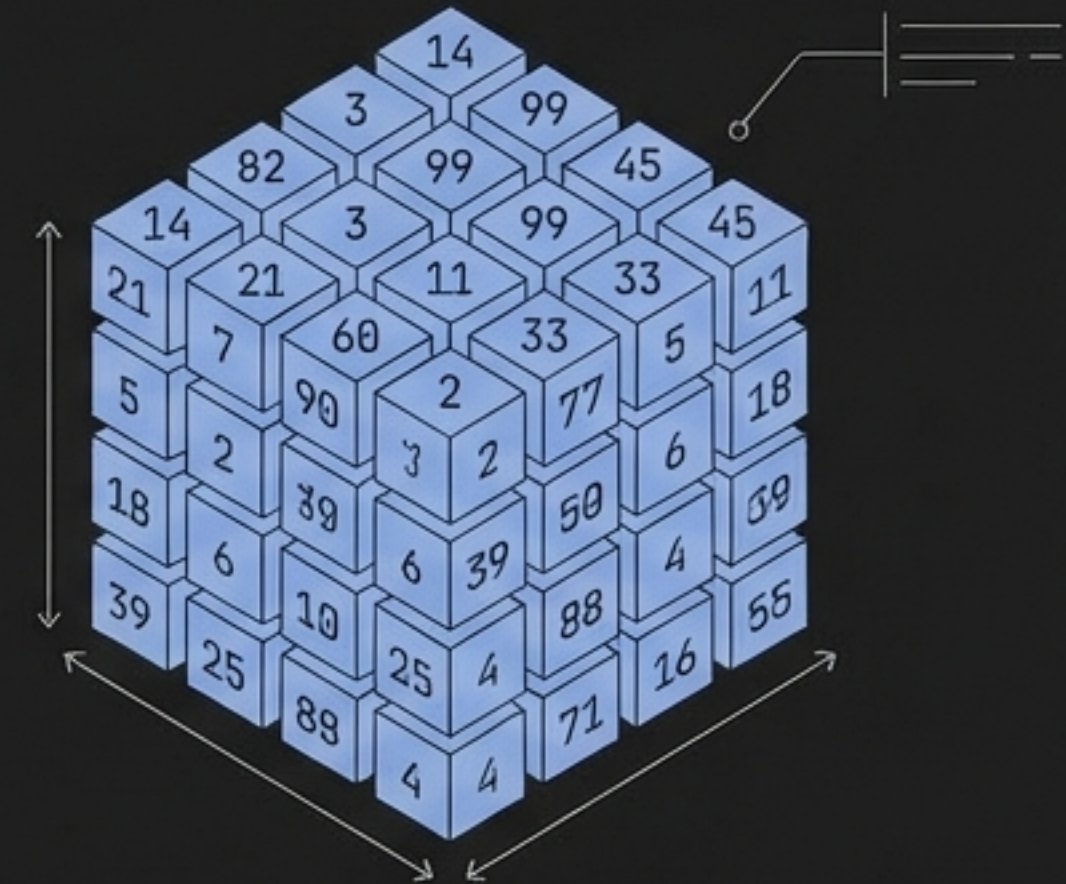


Standardization

A classic Bell Curve centered at 0.

Random Integers

`np.random.randint(low, high, size)`

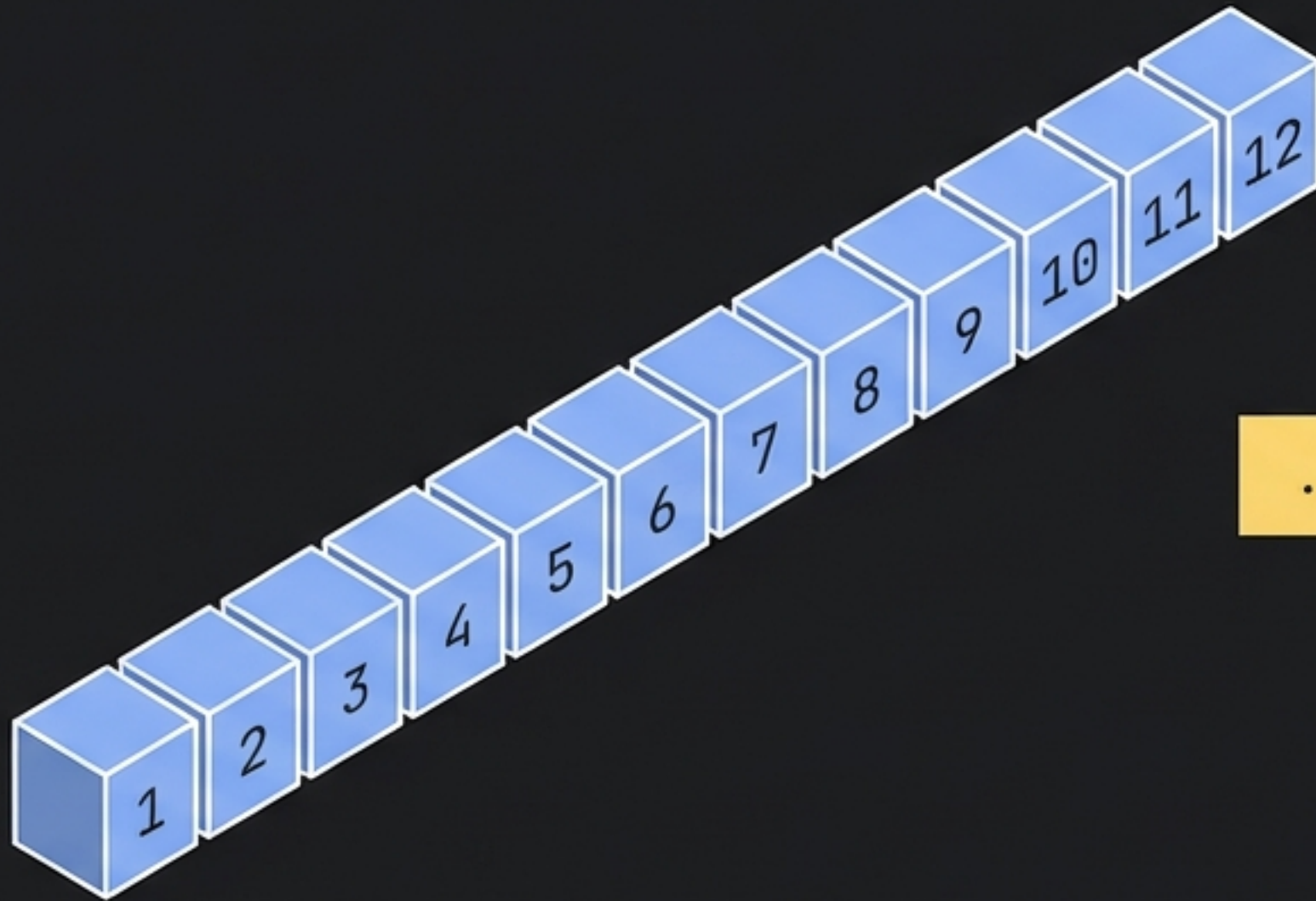


Discrete Selection

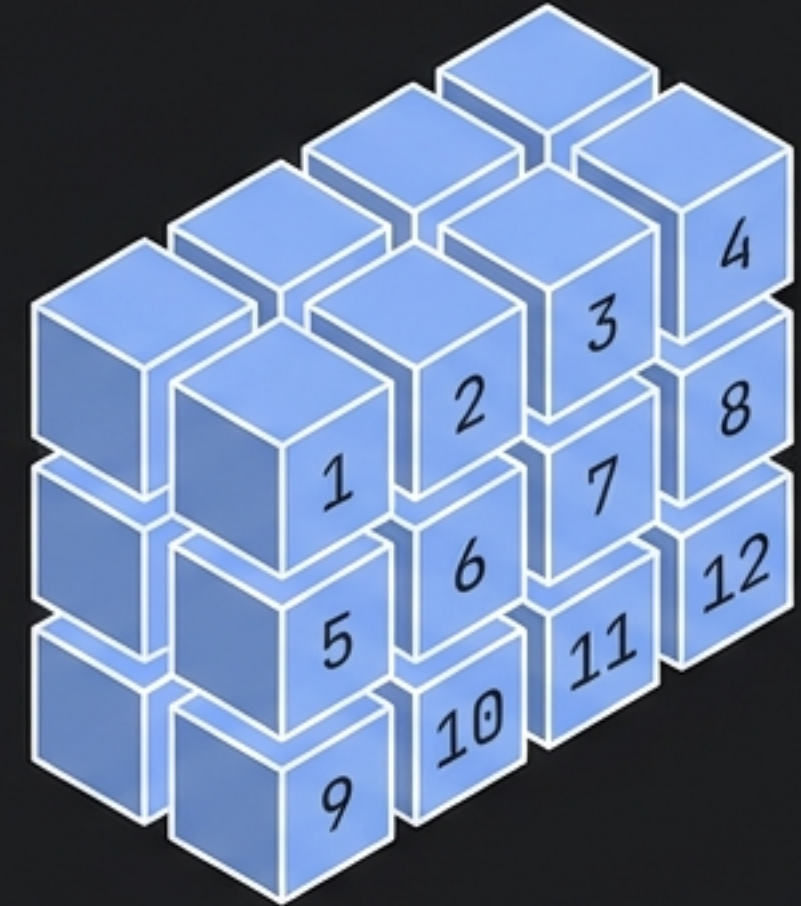
Populated with random whole numbers within a range.

Inspection & Reshaping

Conservation of Mass



`.reshape(3, 4)`



Attributes:

```
.shape = (3, 4)  
.size = 12 (Total elements)  
.dtype = int64
```

⚠ The **Rule of Conservation**: New dimensions product (3×4) must equal total size (12).

Precision Targeting: Indexing & Slicing

	0	1	2	3
0	12	24	36	48
1	52	64	76	88
2	90	92	94	96
3	98	100	102	104

`data[1, 2]`
Row 1, Col 2

`data[0:2, 1:3]`
Block: Rows 0-1, Cols 1-2

`data[:, 1]`
Entire Column 1

Syntax: [Row, Column]

Selects a single element by row and column index.

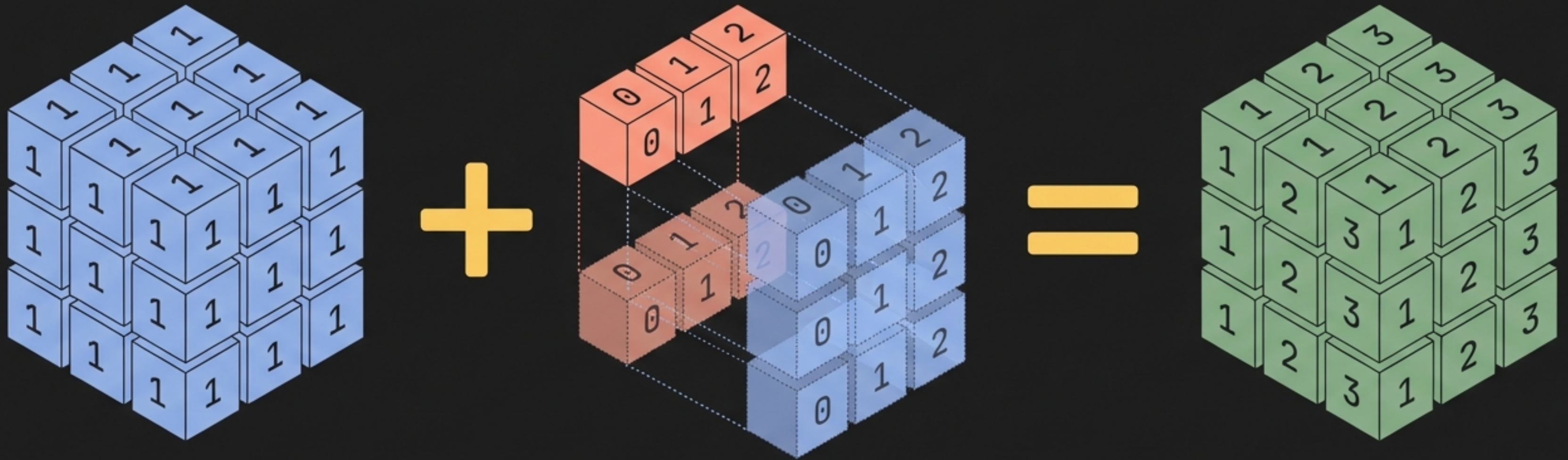
Slicing:

[Start : Stop : Step]

Extracts a subarray by range, with optional step.

The Magic of Broadcasting

Visual demonstration of “Math without Loops.”

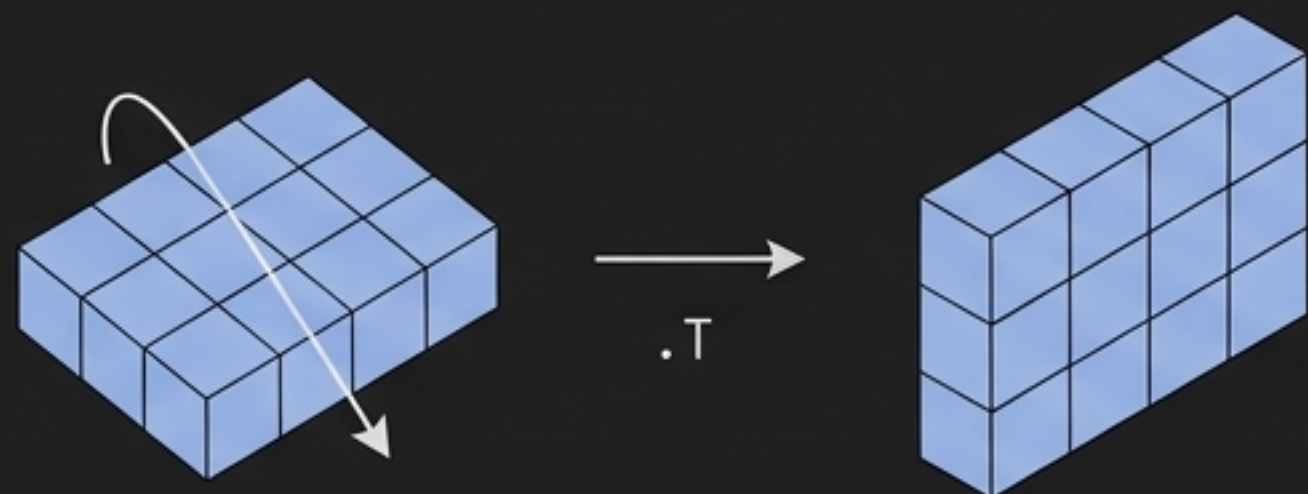
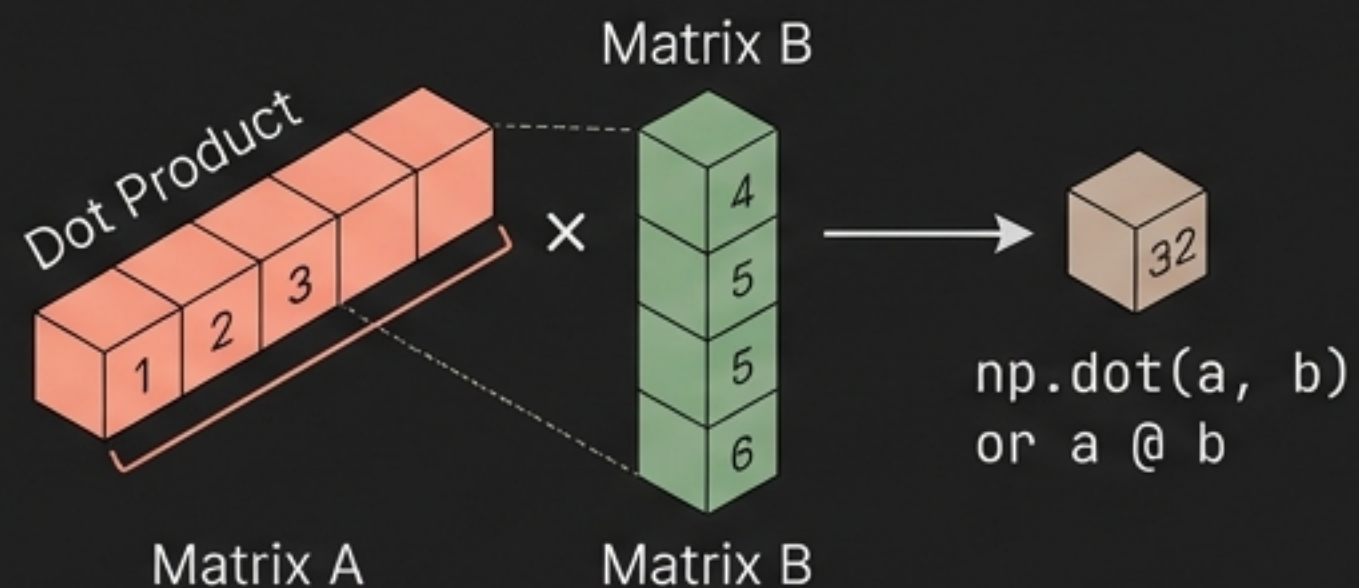


NumPy automatically stretches the smaller array to fit the dimensions of the larger one during arithmetic operations.

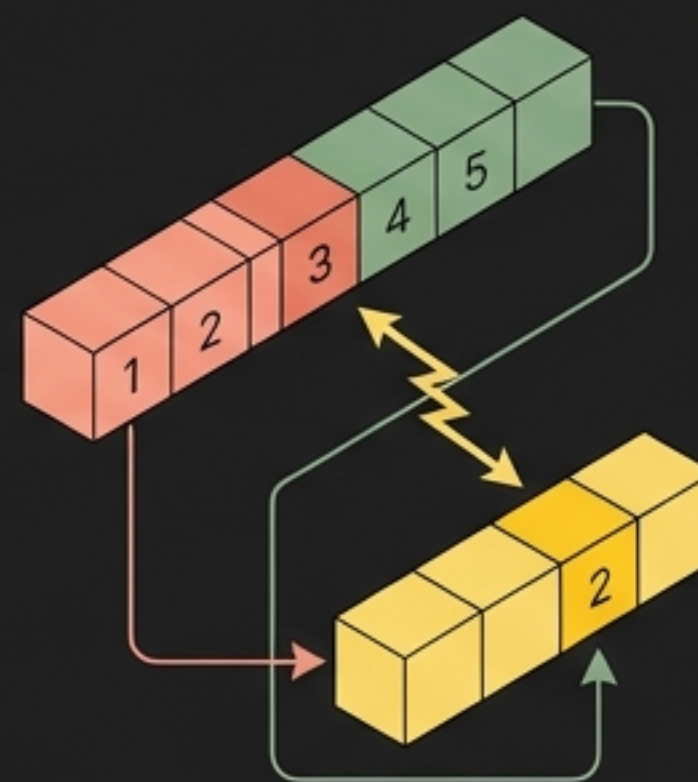
Matrix Operations & The Copy Warning

Visualizing linear algebra and memory management in data manipulation

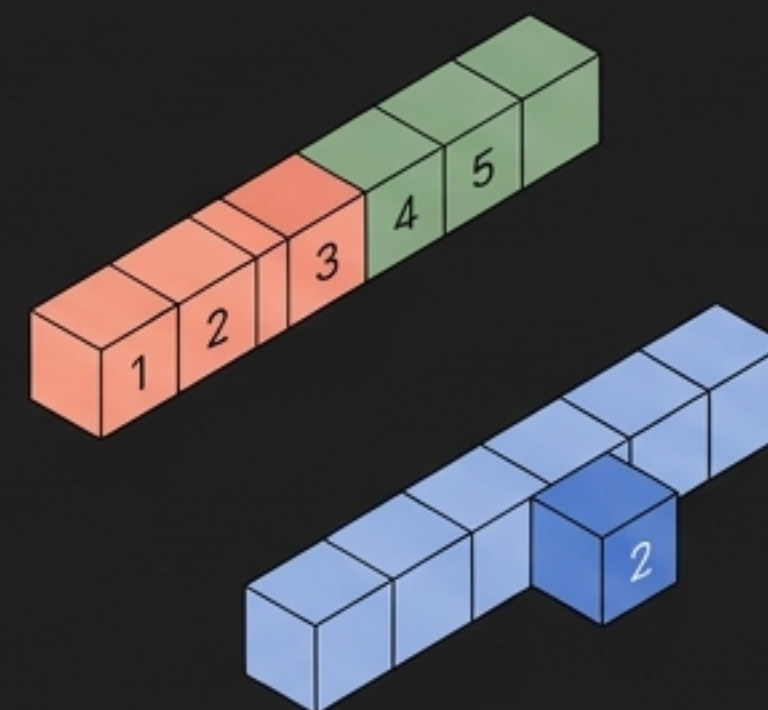
Linear Algebra



The Memory Trap



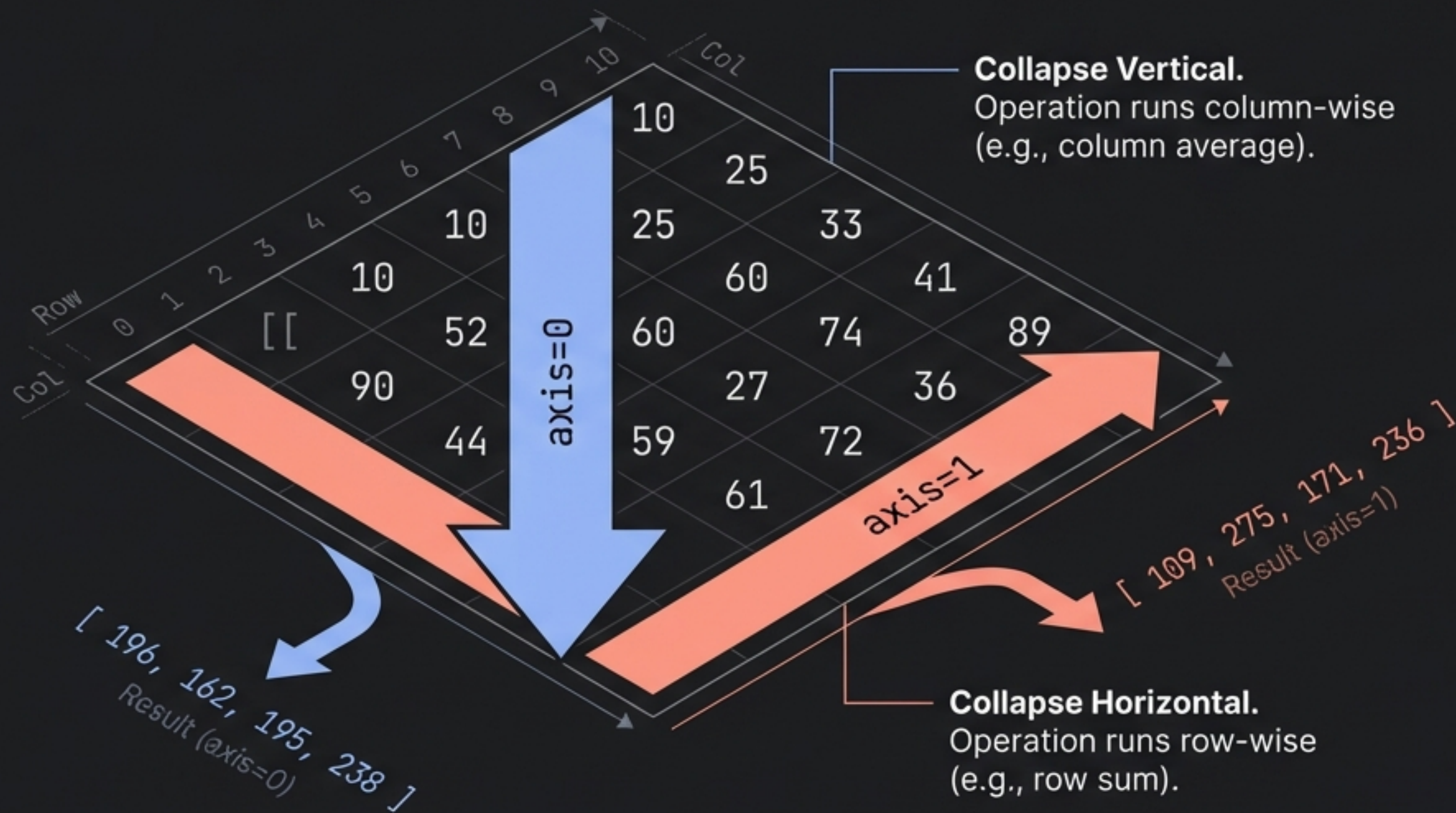
View (Slicing)
Modifies Original



Deep Copy (.copy())
Safe Independent Duplicate

Statistical Aggregation: Understanding Axes

Visualizing column-wise and row-wise operations on data matrices.

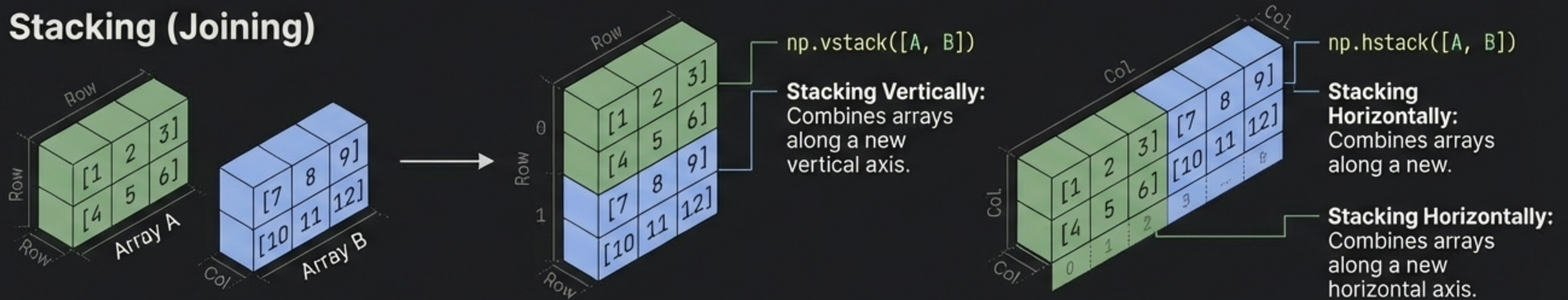


- **.sum()**
Calculates the sum of elements along the specified axis.
- **.mean()**
Calculates the mean of elements along the specified axis.
- **.min()**
Calculates the min of elements along the specified axis.
- **.max()**
Calculates the max of elements along the specified axis.
- **.std()**
Calculates the std of elements along the specified axis.

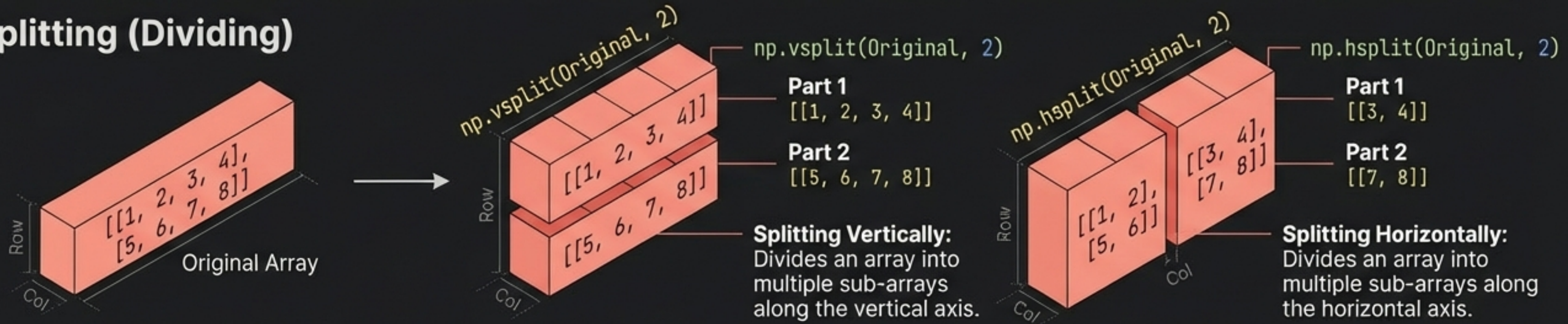
Structural Engineering: Stacking & Splitting

Visualizing array combination and division operations.

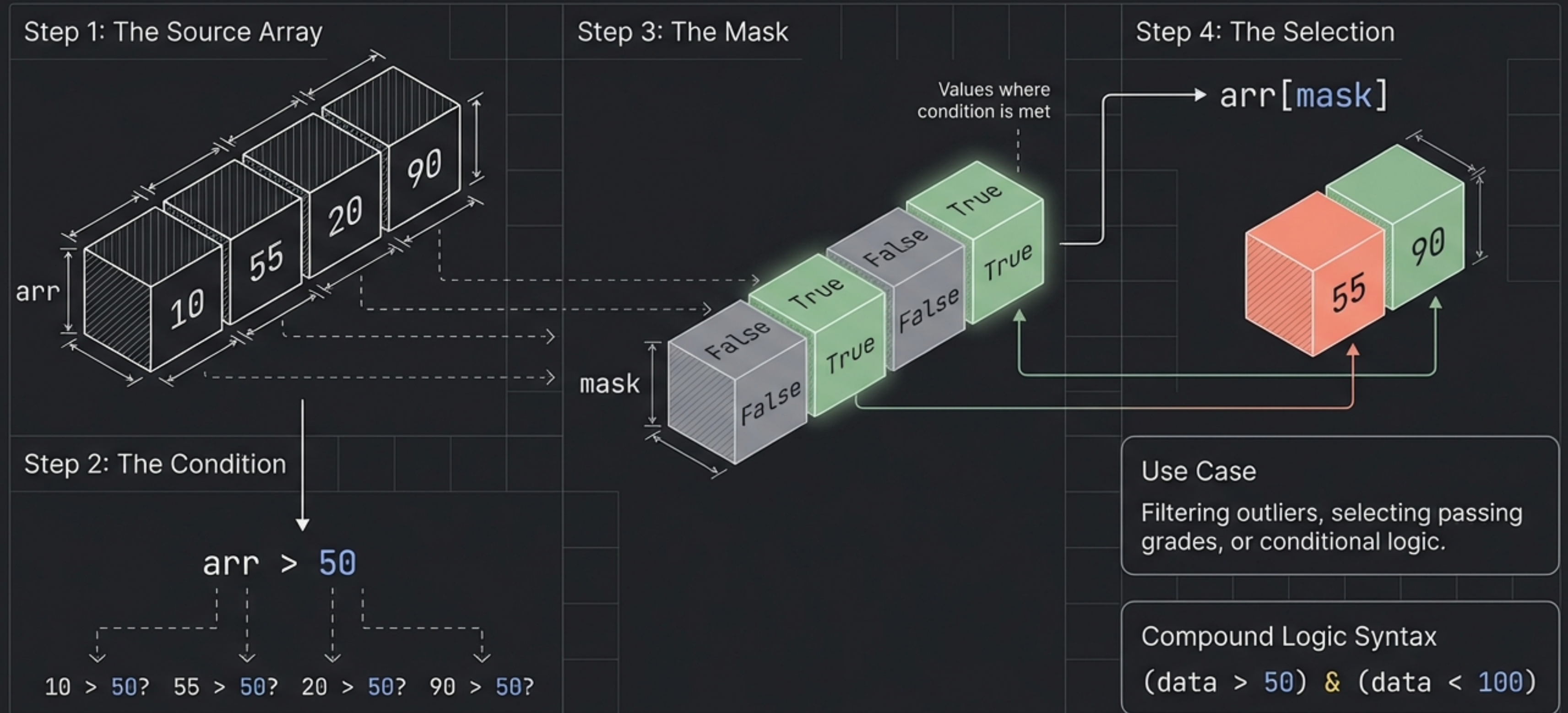
Stacking (Joining)



Splitting (Dividing)

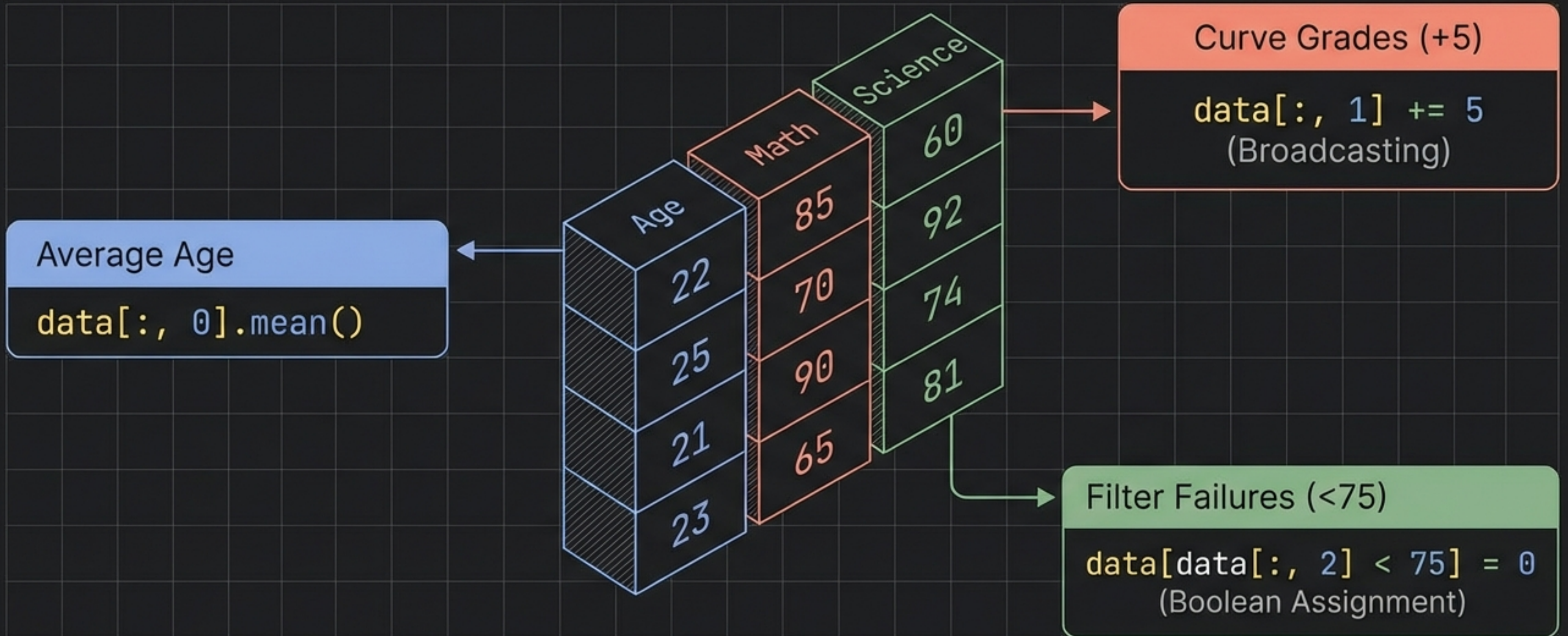


Boolean Indexing: Querying Data



Case Study 1: Student Marks Analysis

Applying NumPy operations to real-world data.



Case Study 2: Sudoku Validator Logic

Row Validation

Row Sum = 45
`np.sum(board, axis=1)`

Column Validation

Col Sum = 45
`np.sum(board, axis=0)`

Col Sum = 45
`np.sum(board, axis=0)`

Block Validation

Block Sum = 45
`grid[i:i+3, j:j+3].sum()`

If all aggregations equal 45, the puzzle is valid.